CS253 Assignment 1: Memory-Efficient Versioned File Indexer

230075_Aditya Prakash

March 10, 2026

1 Introduction

This report details the design and implementation of a memory-efficient versioned file indexer for large text-based log records[cite: 7, 13]. The system processes files incrementally using a fixed-size buffer to ensure that memory usage remains independent of the total file size[cite: 17, 18, 75].

2 Problem Statement

The objective is to build a word-level index—mapping unique alphanumeric tokens to their frequencies—while adhering to strict memory constraints[cite: 7, 19, 23]. The program must support version management and three specific query types: Word Count, Top-K frequent words, and Difference queries between two versions[cite: 34, 40].

3 System Architecture

The implementation follows an object-oriented design consisting of four primary classes as required[cite: 59, 60, 62]:

- **BufferedFileReader**: Manages binary file streams and implements incremental reading into a fixed buffer (constrained between 256 KB and 1024 KB)[cite: 53, 54, 55, 63].
- **Tokenizer**: Extracts alphanumeric tokens from the buffer, handles case-insensitivity, and correctly manages tokens split across buffer boundaries[cite: 19, 20, 57, 64].
- **VersionedIndex**: Stores the mapping of words to frequencies for a specific user-provided version name[cite: 36, 37, 65].
- **QueryProcessor**: An abstract base class that facilitates runtime polymorphism for different analytical operations[cite: 66, 68, 69].

4 Design Requirements Implementation

4.1 Polymorphism and Inheritance

The `QueryProcessor` class serves as the abstract base[cite: 68]. Derived classes—`WordQuery`, `TopKQuery`, and `DiffQuery`—implement the `execute()` virtual function to provide specific analytical logic via dynamic dispatch[cite: 69].

4.2 Memory Optimization

To maximize efficiency, the code utilizes `std::string_view` for chunk processing to avoid unnecessary string copies. The tokenizer directly updates the index, ensuring memory grows only with the number of unique words rather than the total file volume[cite: 76, 77].

4.3 Exception Handling

The program employs `try-catch` blocks to manage runtime errors, such as invalid buffer sizes, missing command-line arguments, or file access issues[cite: 71].

5 Command-Line Interface

The program accepts inputs via command-line arguments as specified in the assignment requirements[cite: 79, 80]:

- `--file`, `--file1`, `--file2`: Path to input data.
- `--buffer`: Size in KB (256-1024).
- `--query`: word — top — diff.
- `--word`: Target token for frequency queries.
- `--top`: Number of results for Top-K queries.

6 Data Storage: `VersionedIndex`

This class acts as the core database for a specific text file. It stores the word frequency counts and associates them with a specific "version" name.

Member Variables:

- `versionName` (`string`): An identifier for the document or dataset (e.g., "v1.0").
- `wordCount` (`unordered_map<string, size_t>`): A hash map that stores each unique word as the key and its frequency (count) as the value, allowing for fast $O(1)$ lookups.

Key Methods:

- `addWord()`: Increments the count of a given word. If the word doesn't exist yet, it adds it with a count of 1.
- `getWordCount()`: Safely retrieves the frequency of a word. Returns 0 if the word is not found.

7 File I/O: `BufferedFileReader`

This class handles reading files in chunks (buffers) rather than loading an entire massive file into memory at once.

Member Variables:

- `file` (`ifstream`): The input file stream.
- `bufferSize` (`size_t`): The size of the memory buffer in bytes.
- `buffer` (`vector<char>`): The actual memory allocated to hold the chunk of data.

Key Methods:

- **Constructor**: Opens the file in binary mode and strictly enforces a buffer size between 256KB and 1024KB (1MB) to balance memory usage and I/O overhead.
- `readChunk()`: Reads a chunk of data into the buffer and outputs it as a `string_view`. Using `string_view` is a crucial optimization because it provides a "window" into the buffer without actually copying the string data into a new variable.

8 Text Processing: `Tokenizer`

This class is responsible for breaking down raw text chunks into individual, clean words and feeding them directly into the `VersionedIndex`.

Key Method:

- `tokenizeAndAdd()`: Iterates through the characters of a chunk. It keeps alphanumeric characters, converts them to lowercase, and groups them into a current word. When it hits a non-alphanumeric character (like a space or punctuation), it pushes the current word into the `VersionedIndex`.
- **Handling chunk splits**: Because a chunk might cut off right in the middle of a word, it uses a leftover string to carry the incomplete word over to the next chunk.

9 The Query Framework (Polymorphism)

The code uses an object-oriented design pattern called the Strategy Pattern to handle different types of queries.

`QueryProcessor` (Abstract Base Class)

This is an interface (a contract) for all queries. It declares a pure virtual function `virtual void execute() = 0;`, meaning any class that inherits from `QueryProcessor` must implement its own version of `execute()`.

`WordQuery` (Derived Class)

Queries the exact frequency of a single specific word.

- **Variables**: A reference to a `VersionedIndex` and the word to search for.
- **execute()**: Prints the version name and the exact count of the word.

`TopKQuery` (Derived Class)

Finds the most frequently used words in the document.

- **Variables**: A reference to a `VersionedIndex` and a number `k` (how many top words to show).
- **execute()**: Copies the hash map data into a vector of pairs, sorts the vector in descending order based on the frequency counts, and utilizes the global `printTopK` template function to display the top results.

`DiffQuery` (Derived Class)

Compares the frequency of a specific word across two different documents (versions).

- **Variables**: References to two different `VersionedIndex` objects (`v1` and `v2`) and the target word.
- **execute()**: Fetches the count from both indexes, subtracts the count of `v1` from `v2`, and prints the difference (which can be negative if the word was used less in the second version).

10 Output Explanation

When you execute the program, it provides a detailed breakdown of the operation. Here is what each part of the output represents:

Version name(s): Displays the identifier (e.g., v1 or v2) assigned to the specific file being processed.

Query result: The core data requested.

- **For Word Query,** it shows the exact frequency of the token.
- **For Top-K,** it lists the most frequent words in descending order.
- **For Difference,** it shows the mathematical change (increase or decrease) in word usage between two versions.

Allocated buffer size: Confirms that the program respected the fixed-size constraints (between 256 KB and 1024 KB) during execution.

Total execution time: Shows the time taken in seconds to read the file, tokenize the text, and process the query.

11 Conclusion

The developed system successfully indexes large log files with a constant memory footprint for the buffer[cite: 7, 13, 56]. By utilizing standard C++ features such as templates, inheritance, and polymorphism, the solution provides a scalable architecture for file version analysis[cite: 8, 68, 69, 72].